CSIGen Documentation

Berkay Guler

Version 0.3.5
Last Updated: October 14th, 2026

Contents

1 Overview **2**
1.1 Quick Start . . . 2
1.2 Conventions . . . 2

2 Scenes **3**
2.1 Generating a Scene . . . 3
2.2 Materials . . . 3
2.3 Available Scenes . . . 3

3 Configuration **6**
3.1 Reference . . . 7

4 How a Run Works **9**

5 Output **11**
5.1 Files . . . 11
5.2 CFR . . . 11
5.3 Per-Sector Metadata . . . 11
5.4 Python API . . . 11

6 Path Data **12**
6.1 CFR from the Paths . . . 12
6.2 Rebuilding and Modifying the CFR . . . 12
6.3 Path Statistics . . . 13

7 Train/Test Splits **14**
7.1 Methods . . . 14
7.2 Output . . . 14

8 Examples **15**

9 Reproducibility and Limitations **16**

1 Overview

CSIGen generates large wireless channel datasets with Sionna RT from a single YAML config. For a 3D scene it places multi-sector base stations on the highest rooftops or on masts at given positions, samples users from a ray-traced radio map, traces the paths from every sector to its users, and saves per sector the channel frequency response (CFR), the paths, and the metadata needed to use and reproduce them.

1.1 Quick Start

Install CSIGen with pip from the root of the repository, with the extra `rt` for channel generation. It installs Sionna RT 2.2.0 and Mitsuba 3.9.1, and a CUDA GPU is strongly recommended. Then run a config from the repository root with the command `csigen-run`. The example configs give the scene files relative to the repository root.

```
pip install -e ".[rt]"
csigen-run --config config/examples/urban/la_28ghz.yaml
```

The base install (`pip install -e .`) needs only NumPy, PyYAML and pydantic. It is enough to read saved path data (Section 6), to validate configs and to split runs, without Sionna RT or a GPU. CSIGen needs Python 3.12 or later. The extra `rt` pins Sionna RT and Mitsuba. The other dependencies are lower bounds, so pip can install newer versions. The README lists the tested versions and the command that installs them. The run writes `output/<scene_name>/<YYYYmmdd_HHMMSS>/` with one `channel_tx_<t>.npz` per sector and a `metadata.yaml` (Section 5). Load a file with:

```
import numpy as np
z = np.load("output/la_1/<run_id>/channel_tx_0.npz", allow_pickle=True)
h = z["h"]                          # [users, rx_ant, 1, tx_ant, ofdm_symbols, subcarriers], complex64
meta = z["metadata"].item()         # positions, LoS labels, path statistics, config, ...
```

The example configs are in `config/examples/`, in one folder per scene type (`urban/`, `suburban/` and `rural/`). Each file name gives the city or town and the carrier, for example `boston_3p5ghz.yaml` at 3.5 GHz and `boston_28ghz.yaml` at 28 GHz. Boston, Chicago, New York and San Francisco have configs at both carriers. Los Angeles has a config at 28 GHz only, and the suburban and rural scenes at 3.5 GHz only. To split a run into train and test runs, see Section 7.

1.2 Conventions

- **Names and indices.** Base station b has sectors `BS_<b>_sector_<s>`, $s = 1, \ldots, N_s$ for $N_s =$ `num_sectors`; their TX index is $t = bN_s + s - 1$, which numbers the output files. Users are `UE_<k>`, numbered TX by TX: the users sampled for TX t follow those of TX $t - 1$.
- **Cells.** A *cell* is a triangle of the measurement-surface mesh on which the radio map is computed (Section 4), not a cell of the cellular network. The coverage area of a base station antenna is called a *sector*.
- **Units.** Meters, seconds, Hz and radians, except for the config, which gives angles in degrees. Element spacings are in carrier wavelengths. Positions and angles are in the scene's global frame.
- **CFR scale.** `h` is the unitless channel gain (power gain $|h|^2$, about the path loss), not scaled by the TX power: `tx_power_dbm` enters only the radio-map metrics `rss` and `sinr` used for user sampling. The received power on a subcarrier is $P|h|^2$, or $P|\mathbf{hw}|^2$ with a unit-norm precoder $\mathbf{w}$, for the transmit power P you assign to that subcarrier. With `cfr_normalize: true`, `load_cfr(file, normalize=False)` recovers the unnormalized CFR from the saved paths, and `path_stats` holds each channel's `path_gain_db`.

2 Scenes

2.1 Generating a Scene

Scenes are made with the `scenegen` tool of Geo2SigMap from a longitude/latitude bounding box (bboxfinder.com helps to pick one):

```
scenegen bbox -71.0602 42.3512 -71.0484 42.3591 --data-dir scenes/sample_scene --enable-lidar-terrain
```

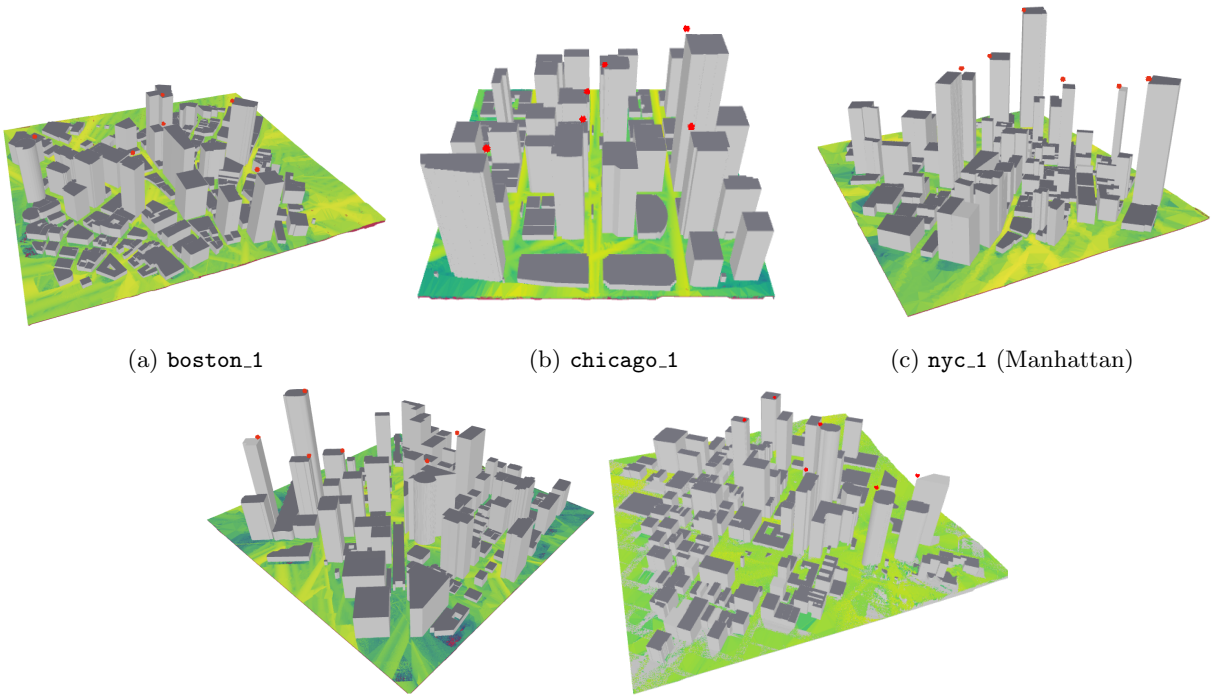
The folder gets `scene.xml`, which references `.ply` meshes under `mesh/` and stores the bounding box. Without `--enable-lidar-terrain`, the ground is a flat plane. CSIGen needs the scene's buildings, named `building_<x>_rooftop` and `building_<x>_wall`, and the terrain object `ground`, on which users are placed. Load scenes with `load_scene(path, merge_shapes=False)`; merged shapes lose the per-building objects.

2.2 Materials

Buildings and ground use ITU materials (mostly `concrete` and `metal`). The ITU ground models of the scene files (`wet_ground` and others) are defined for 1–10 GHz only, so above 10 GHz set `override_ground_material`. All materials have Sionna's default scattering coefficient of 0, so the scenes produce no diffuse paths, even with diffuse reflection enabled.

2.3 Available Scenes

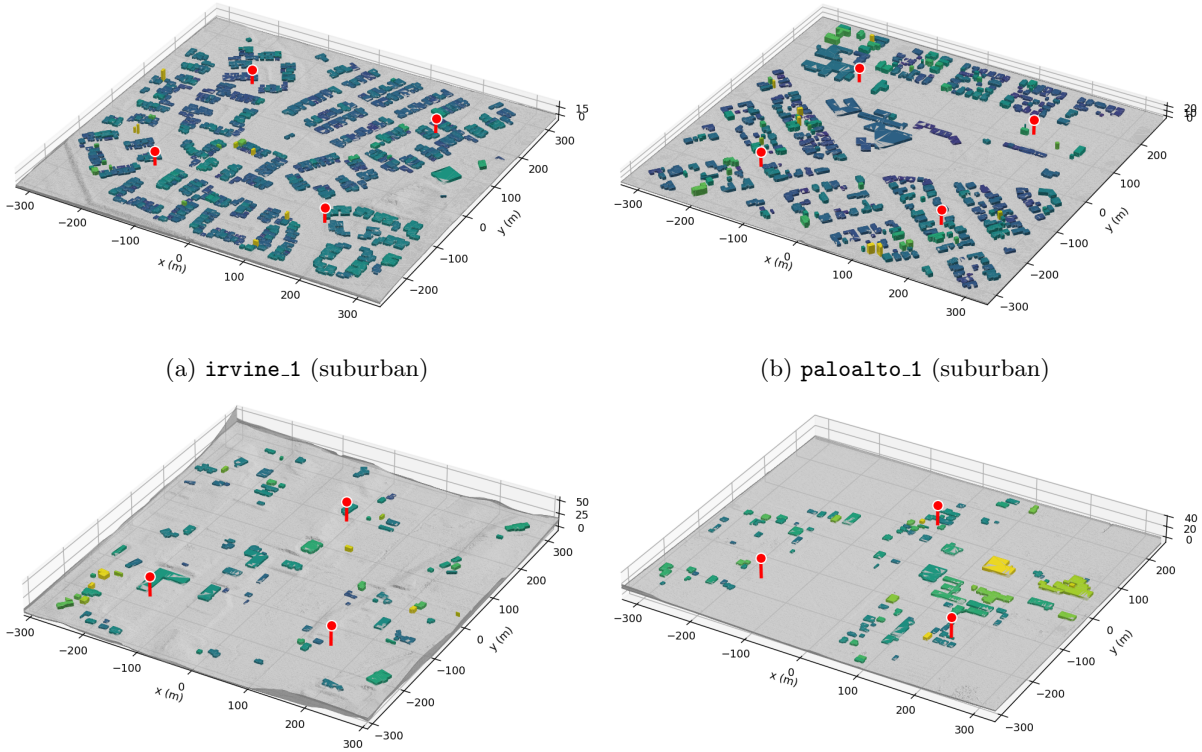
The repository includes nine scenes under `scenes/<name>/scene.xml`. Five are downtown scenes (Fig. 1). Two are suburban scenes and two are rural scenes (Fig. 2). The suburban scenes (`irvine_1`, `paloalto_1`) are residential areas with mostly one- and two-story houses. The rural scenes (`aguadulce_1`, `littlerock_1`) have about 100 scattered buildings each. These scenes have no tall buildings for the rooftop placement. Thus, their example configs put the base stations on masts with `bs_positions` and `bs_mast_height`. The suburban configs use four masts of 25 m, the urban macro (UMa) default of TR 38.901 V17.0.0, which has no suburban scenario. The rural configs use three masts of 35 m, the rural macro (RMa) default. Each of these scene folders also holds `scene_source.yaml`, which records how `scenegen` made the scene. It gives the center, the size, the LiDAR settings for terrain and building heights, and the materials.



(a) `boston_1` (b) `chicago_1` (c) `nyc_1` (Manhattan)

(d) `sf_1` (e) `la_1`

Figure 1: Included downtown scenes (Boston, Chicago, New York, San Francisco and Los Angeles).



(a) `irvine_1` (suburban)

(b) `paloalto_1` (suburban)

(c) `aguadulce_1` (rural)

(d) `littlerock_1` (rural)

Figure 2: Included suburban and rural scenes, at true vertical scale, with the base stations of the example configs (red, on masts).

3 Configuration

A run is fully defined by one YAML file. All keys are required except those marked optional below; `load_validated_config(path)` (`csigen.config_validator`) checks types, ranges and combinations before anything runs. A complete example:

config.yaml

```yaml
# Scene
scene_xml_path: "./scenes/boston_1/scene.xml"
carrier_frequency: 3500000000.0  # Hz

# Scene setup
scene_center: [0.0, 0.0]          # [x, y] (m); BS antennas are placed toward it
antenna_height_offset: 10.0       # BS antenna height above the rooftop (m)
num_deployment_buildings: 1       # number of base stations, on the highest buildings
# bs_positions: [[-150, -150], [150, 150]]  # optional: base stations on masts at [x, y] (m) instead
# bs_mast_height: 25.0                      # optional, with bs_positions: antenna height above terrain (m)
clip_terrain_to_buildings: true   # clip the ground to the building area
terrain_clip_margin: 15.0         # margin around the buildings when clipping (m)
user_shift_from_ground: 1.5       # UE height above ground (m)
# override_ground_material: "asphalt_concrete"  # optional: "concrete" or "asphalt_concrete"

# TX (base station) array; RX (user) array with the same keys and rx_ prefix
tx_num_rows: 4
tx_num_cols: 8
tx_vertical_spacing: 0.5          # wavelengths
tx_horizontal_spacing: 0.5        # wavelengths
tx_pattern: "tr38901"             # tr38901, dipole, iso or hw_dipole
tx_polarization: "V"              # V, H, VH or cross (±45°)
rx_num_rows: 1
rx_num_cols: 1
rx_vertical_spacing: 0.5
rx_horizontal_spacing: 0.5
rx_pattern: "iso"
rx_polarization: "V"

# Base stations
num_sectors: 6                    # sectors per base station, evenly spread in azimuth
mechanical_tilt: 10.0             # downtilt (degrees, positive = down)
azimuth_offset: 0.0               # azimuth of the first sector (degrees)
tx_power_dbm: 43.0                # per sector; used for the rss/sinr sampling metrics only

# Radio map (user sampling)
radio_map_specular_reflection: true     # mirror-like reflections
radio_map_diffuse_reflection: false     # scattering from rough surfaces (needs a scattering coefficient > 0)
radio_map_refraction: true              # transmission through materials
radio_map_diffraction: true             # diffraction on building edges (wedges)
radio_map_edge_diffraction: true        # also on free-floating edges
radio_map_diffraction_lit_region: false # also where the direct path is not blocked
radio_map_max_depth: 5                  # maximum interactions per path
radio_map_samples_per_tx: 1000000       # rays per sector
radio_map_seed: 1

# User sampling
num_user_samples_per_tx: 200      # users per sector (at most)
user_sample_seed: 1
user_sample_min_val_db: -150      # metric thresholds (dB; dBm for rss)
user_sample_max_val_db: 100
user_sample_min_dist: 0.0         # distance to the TX (m)
user_sample_max_dist: 10000.0
user_sample_metric: "path_gain"   # path_gain, rss or sinr
tx_association: true              # only radio-map cells (mesh triangles) where this TX is the strongest
sample_center_pos: true           # users at cell centres (else random in the triangle)
scene_edge_epsilon: 1.0           # drop users this close to the edge of the sampled area (m; 0 = off)

# Mobility: mobility_preset names one of the presets below
mobility_preset: "vehicular"
mobility_presets:
```

```yaml
  stationary:       {orientation_mode: "random"}
  stationary_to_tx: {orientation_mode: "to_tx"}
  pedestrian:       {orientation_mode: "random", speed_distribution: "pedestrian", direction_mode: "random"}
  pedestrian_to_tx: {orientation_mode: "to_tx", speed_distribution: "pedestrian", direction_mode: "random"}
  vehicular:        {orientation_mode: "random", speed_distribution: "vehicular", direction_mode: "random"}
  slow_walking:     {orientation_mode: "random", speed_distribution: "uniform_continuous",
                     speed_min: 0.5, speed_max: 1.5, direction_mode: "random"}
  fast_walking:     {orientation_mode: "random", speed_distribution: "uniform_continuous",
                     speed_min: 1.5, speed_max: 2.5, direction_mode: "random"}

# Path solver (channels)
path_solver_max_depth: 5                    # maximum interactions per path
path_solver_max_num_paths_per_src: 10000000 # path buffer size per source
path_solver_samples_per_src: 10000000       # rays per source
path_solver_synthetic_array: true           # trace from array centres, add element phases
path_solver_los_mode: true                  # line-of-sight paths
path_solver_specular_reflection: true       # interaction toggles as for the radio map
path_solver_diffuse_reflection: false
path_solver_refraction: true
path_solver_diffraction: true
path_solver_edge_diffraction: true
path_solver_diffraction_lit_region: false
path_solver_seed: 1
path_solver_per_tx_users_only: false  # true: each sector only its own users
path_solver_rx_batch_size: 50         # optional (default 50); receivers per solve (null = all at once)
path_solver_spec_table_size: 400000000  # optional (default 4e8 with a synthetic array); Sionna's table size
↪  (null = Sionna default)

# OFDM and CFR
num_subcarriers: 32
num_ofdm_symbols: 14
subcarrier_spacing: 30000.0       # Hz
cfr_normalize_delays: false       # subtract the first delay of each channel
cfr_normalize: false              # unit average energy per channel
cfr_out_type: "numpy"             # used only without a synthetic array

# Saved data (optional)
save_paths: true                  # default: true with a synthetic array
save_cfr: true                    # default: true

# Receivers of an earlier run (optional)
# receivers_from: output/boston_1/20261015_120000   # solve its users again instead of sampling users
```

3.1 Reference

Only what the comments above do not say is listed here.

Scene setup. `num_deployment_buildings` is the number of base stations (Section 4). With `bs_positions` (optional, a list of $[x, y]$ positions in meters) and `bs_mast_height` (meters), the base stations stand on masts instead: one at each position, its antennas `bs_mast_height` above the terrain there, and `num_deployment_buildings` and `antenna_height_offset`, otherwise required, can be left out. This suits scenes whose highest buildings are houses, such as suburban or rural areas. Give both keys or neither. The positions must lie inside the clipped terrain (with `clip_terrain_to_buildings`), and an antenna must not be below the top of a building whose bounding box contains its position. `override_ground_material` (optional) assigns `concrete` (1–100 GHz) or `asphalt_concrete` (1–40 GHz) to the `ground` object and removes the wet-ground materials; without it, the scene's materials are kept.

Arrays. All sectors share the TX array and all users the RX array (Sionna RT allows one of each per scene). `VH` and `cross` have two polarizations, so twice the elements. The antenna order is given in Section 5.

Interaction toggles. The `radio_map_*` and `path_solver_*` flags (reflection, refraction, diffraction, depth, samples, seed) are separate, so the radio map, which only places users, can be cheaper than the path

solve. See the Sionna RT technical report for their meaning. Diffuse reflection has no effect with the included scenes (Section 2.2).

User sampling. `num_user_samples_per_tx` is a maximum: a sector with fewer valid cells gets fewer users, possibly none. The radio-map metric is per sector.

Mobility. `mobility_preset` must be one of the seven preset names shown. `orientation_mode`: `random` (yaw uniform, pitch within $\pm 30°$, roll within $\pm 45°$) or `to_tx` (the array faces the user's sector). `speed_distribution`: none (stationary), `pedestrian` (0.5–2 m/s), `vehicular` (8–30 m/s) or `uniform_continuous` between `speed_min` and `speed_max`; the direction is uniform in the horizontal plane. Users do not move during a run; the velocity only sets the Doppler shifts.

Path solver. `path_solver_synthetic_array: true` traces rays from the array centres only and adds the phase of each element analytically; it is much faster and required for the path data (Section 6). `path_solver_per_tx_users_only: false` solves every sector for all users of the scene, so each user appears in every sector's file. `path_solver_rx_batch_size` and `path_solver_spec_table_size` avoid silent path loss (Section 4); their defaults are 50 and $4 \cdot 10^8$. The table space per receiver is about the table size divided by the batch size, $8 \cdot 10^6$ at the defaults. To save GPU memory, lower both keys by the same factor, so that this ratio stays the same. A smaller batch then needs less memory but makes the run slower. For example, the suburban and rural example configs use 25 and $2 \cdot 10^8$. Without a synthetic array, the table keeps Sionna's default (10^6), since every TX antenna is a source; batching then needs `cfr_out_type: numpy`, so with another output type the receivers are solved at once, with a warning. The validator rejects settings whose table indices exceed Sionna's 32-bit range: max(table size, `max_num_paths_per_src`) times the number of sources (1 with a synthetic array, else the number of TX antennas) must stay below 2^{32}.

CFR. `cfr_out_type` (`numpy`, `tf`, `torch`, `jax` or `drjit`) applies only without a synthetic array, where Sionna computes the CFR; with batching it must be `numpy`.

Saved data. `save_paths` requires a synthetic array. With `save_cfr: false`, the files hold only the paths, from which `load_cfr` rebuilds the CFR. At least one of the two must be true.

Receivers of an earlier run. `receivers_from` (optional) is the directory of an earlier run, with its `channel_tx_*.npz` files. Its users are solved again, with their names, positions, velocities and orientations, instead of sampling new ones, so that the two runs hold channels of the same users, for example at two carrier frequencies or with two arrays. The scene and the base stations must be those of the earlier run: the same scene contents (compared by `scene_sha256`, or by the earlier run's scene file for runs before 0.3.6), the same scene-setup and base-station keys, and the same sectors at the same positions and orientations; any difference is an error. The radio map, user sampling and mobility keys are still required by the validator but ignored, and a warning says so. The earlier run must be a full run of CSIGen 0.3.3 or later, not the output of the split script. Users that had no path to any sector in the earlier run are not in its files and are left out; `users_per_tx` and `total_users` in the metadata stay those of the earlier run, so that the user indices and serving sectors keep their meaning.

4 How a Run Works

`csigen-run` calls `generate_channels(config)` (`csigen.channel_generator`), which runs these steps.

1. Scene setup. The scene is loaded at `carrier_frequency`. With `clip_terrain_to_buildings`, the ground is clipped to the bounding box of the buildings plus `terrain_clip_margin`, so that users are not placed in empty land around the city (Fig. 3). The clipped ground, raised by `user_shift_from_ground`, is the measurement surface on which users are placed.

2. Base stations. With `bs_positions`, each position gets one base station on a mast: its height is that of the terrain triangle containing the position, interpolated within the triangle and taken before the clipping, plus `bs_mast_height`; a position inside the bounding box of a building logs a warning, or is an error if the antenna would be below the building's top. Otherwise, the `num_deployment_buildings` buildings with the highest centroids each get one base station, at the rooftop vertex closest to `scene_center` (so that the building lies behind the antenna) and `antenna_height_offset` above the roof's highest point (Fig. 4a). Each base station has `num_sectors` sectors at the same position, with azimuths `azimuth_offset` $+ 360°(s-1)/N_s$ and the downtilt `mechanical_tilt`; the orientation in Sionna's angles is $[\text{azimuth}, \text{tilt}, 0]$.

3. Radio map and user sampling. A radio map over the measurement surface gives `user_sample_metric` per sector in every cell, a triangle of the surface mesh. A cell is valid for a sector if the metric lies within the thresholds, the cell centre lies within the distance limits and, with `tx_association`, the sector is the strongest in the cell. Each sector takes up to `num_user_samples_per_tx` distinct valid cells (Fig. 4b shows an example): every cell gets a random key from `user_sample_seed`, and the sector takes the valid cells with the lowest keys. The radio map is not bit-identical between GPU runs, and with this rule a cell that changes validity moves at most one user. Users are placed at the cell centres or, with `sample_center_pos: false`, at a random point of the cell.

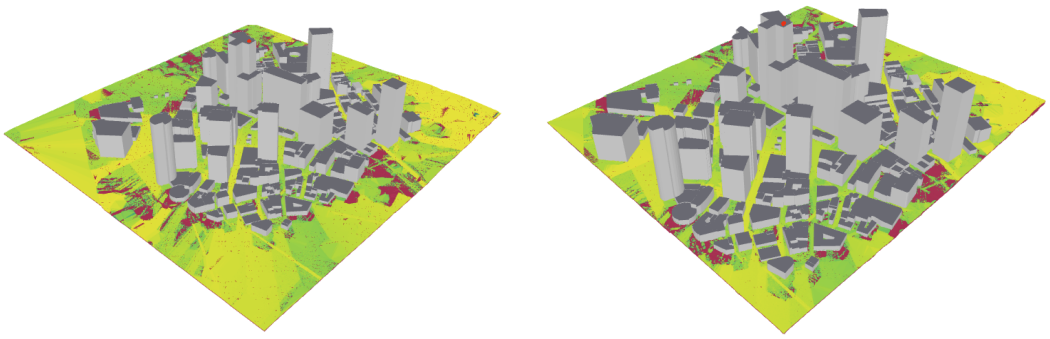
4. Edge filtering. With `scene_edge_epsilon` > 0, users within that distance of the bounding box of all sampled users are removed (Fig. 5). These users tend to be LoS-dominated. The filter acts after sampling, so a sector can keep fewer users than requested.

5. Receivers. Each user becomes a receiver with the orientation and velocity of the mobility preset, drawn with `user_sample_seed`. With `receivers_from`, steps 3–5 (radio map, edge filtering, receivers) are replaced: the receivers of the earlier run are added as they were, after a check that its scene and base stations match.

6. Path solving. Each sector is solved alone, for its own users or for all users. Sionna RT stores each new chain of reflecting surfaces per receiver in a hash table of fixed size per source; when many receivers share one solve, the table fills up and new chains are discarded without a warning, so some NLoS paths can be lost (LoS paths are traced separately and are kept). CSIGen therefore solves the receivers in batches of `path_solver_rx_batch_size` with a larger table (`path_solver_spec_table_size`). It also records the largest fill of Sionna's path buffer (`path_buffer_fill`, as a fraction of `path_solver_max_num_paths_per_src`) and logs a warning when it reaches 1, that is, when candidate paths were discarded; there is no warning below that.

7. Valid channels and LoS labels. A user without any path of positive delay has no channel to the sector and is dropped from its file. A kept channel is LoS (`los_binary` $= 1$) if it has a path without any interaction.

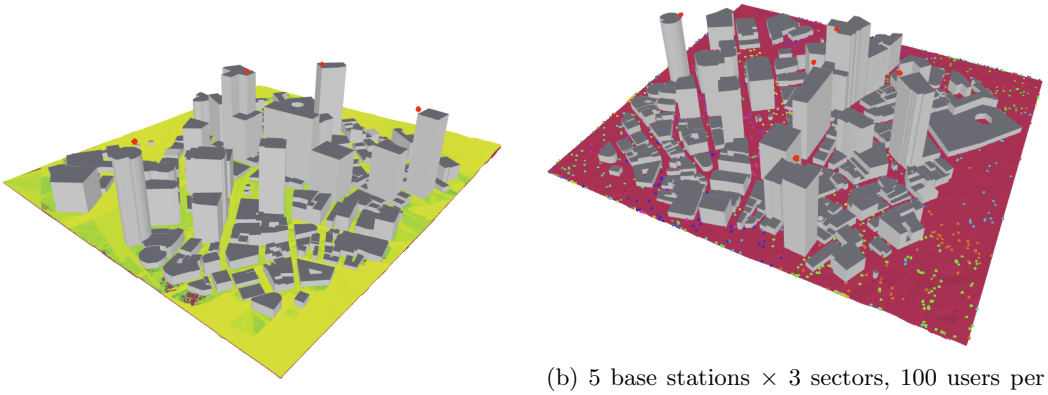
8. Paths and CFR. With a synthetic array, the paths of every kept channel are collected (Section 6), and the CFR is computed from them. Without one, Sionna's `Paths.cfr` computes the CFR and no paths are saved. The results are streamed: `generate_channels` yields one item per sector, and `csigen-run` writes its file before the next sector is solved.



(a) Before clipping

(b) After clipping (15 m margin)

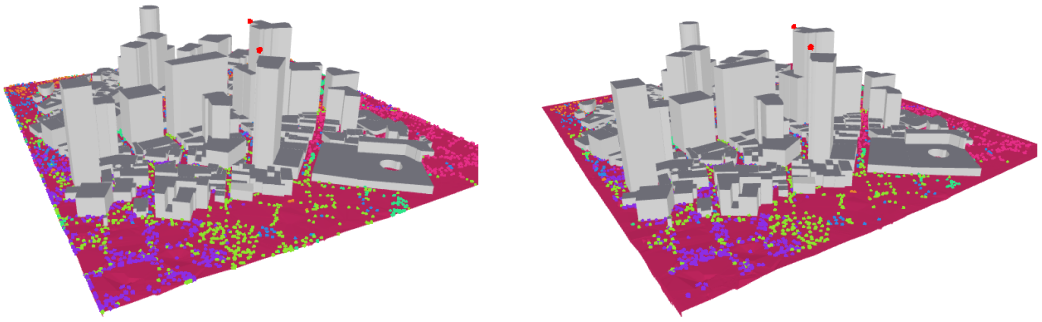
Figure 3: Terrain clipping.



(a) Four base stations (red)

(b) 5 base stations × 3 sectors, 100 users per sector

Figure 4: Base station placement and user sampling.



(a) `scene_edge_epsilon` = 0

(b) `scene_edge_epsilon` = 15 m

Figure 5: Edge filtering, with 3 sectors, 2 base stations and 1000 users per sector.

5 Output

5.1 Files

- `channel_tx_<t>.npz`, one per sector t (compressed): `h`, `shape` and `dtype` if the CFR is saved, the `paths_*` arrays if the paths are saved (Section 6), and `metadata`, a dictionary (load with `allow_pickle=True` and `.item()`).
- `metadata.yaml`: CSIGen version, scene name, timestamp, number of sectors and users, users per sector, CFR shape and dtype per sector, CFR axes, array layouts, and the full validated config.

5.2 CFR

`h` is complex64 with axes `cfr_axes` = (`rx`, `rx_ant`, `tx`, `tx_ant`, `ofdm_symbol`, `subcarrier`): one row per kept user, a TX axis of size 1, and subcarriers ordered from the lowest to the highest frequency. The metadata `tx_array` and `rx_array` give each array's rows, columns, polarizations, spacing, pattern and antenna order: antenna $i = p\,R\,C + c\,R + r$ for row r (from the top), column c (along the array's local y axis) and polarization p, with R rows and C columns. The vertical index therefore runs fastest; reshape an antenna axis to $[P, C, R]$ to get the array.

5.3 Per-Sector Metadata

Arrays marked "per row" are aligned with the rows of `h`.

Key	Content
`tx_idx`, `tx_name`, `tx_position`, `tx_orientation`	the sector; orientation in Sionna's angles $[\alpha, \beta, \gamma]$ (rotations about z, y, x)
`rx_names`, `rx_positions`	per row: the user and its position ($[N, 3, 1]$)
`rx_velocities`, `rx_orientations`	per row: velocity (m/s) and orientation, $[N, 3]$
`rx_serving_tx`	per row: the sector the user was sampled for
`los_binary`	per row: 1 if the channel has a LoS path
`path_stats`, `path_stats_fields`	per row: path statistics (Section 6.3) and their description
`num_valid_channels`	number of rows
`num_txs`, `num_sectors`, `users_per_tx`, `total_users`	the run: sectors, sectors per base station, users sampled per sector, all users
`path_buffer_fill`	largest fill of Sionna's path buffer (at 1, paths were discarded)
`cfr_saved`, `cfr_shape`, `cfr_dtype`, `cfr_axes`	the CFR, also when `h` is not saved
`cfr_params`, `cfr_source`	what `load_cfr` needs; how the CFR was computed
`scene_sha256`	SHA-256 of the scene XML and the mesh files it references (since 0.3.6)
`paths_saved`, `path_fields`	whether paths are saved; their description
`tx_array`, `rx_array`	array layouts and antenna order
`config`	the validated config

5.4 Python API

`generate_channels(config)` is an iterator with one dictionary per sector: `tx_idx`, `tx_name`, `h_tx` (`None` with `save_cfr: false`), `path_data` (`None` without saved paths) and `tx_metadata`. `save_channel_data` (`csigen.channel`) writes one item as a file.

6 Path Data

With a synthetic array, every path has one delay, Doppler shift and pair of directions for all antenna pairs, and CSIGen saves all valid paths of every channel. The paths of a file are stored in a flat layout: the paths of row i are entries `paths_offsets[i]` to `paths_offsets[i+1]` -1 of the other arrays, sorted by power (the mean of $|a|^2$ over antenna pairs), strongest first. The metadata `path_fields` describes each array.

Key	Shape and type	Content
`paths_offsets`	$[N+1]$ int64	start of the paths of each row
`paths_a`	$[P, N_{\mathrm{rx}}, N_{\mathrm{tx}}]$ complex64	coefficient per antenna pair
`paths_tau`	$[P]$ float32	absolute delay (s)
`paths_doppler`	$[P]$ float32	Doppler shift (Hz)
`paths_theta_t`, `paths_phi_t`	$[P]$ float32	zenith and azimuth of departure
`paths_theta_r`, `paths_phi_r`	$[P]$ float32	zenith and azimuth of arrival
`paths_interactions`	$[P, D]$ uint8	interaction type per bounce

P is the number of paths of the file, N_{rx} and N_{tx} the numbers of antennas (in the order of the CFR) and $D =$ `path_solver_max_depth`. The coefficients are passband, unitless and include the antenna patterns. Interaction codes: 0 none, 1 specular reflection, 2 diffuse reflection, 4 refraction, 8 diffraction; 0 also fills the bounces after the last, so a LoS path is all zeros. A path takes about $1/(N_{\mathrm{sym}}N_{\mathrm{sc}})$ of the size of its channel's CFR, so the paths of a channel with few paths (in our runs, medians of 3–11) take a few percent of the CFR; with thousands of paths, as from diffuse scattering, they exceed it.

6.1 CFR from the Paths

CSIGen computes the saved CFR from these arrays, in float64, with `cfr_from_paths` (`csigen.path_data`), for every antenna pair:

$$h(f,t) = \sum_l a_l \, e^{-j2\pi(f_c+f)\tau_l} \, e^{j2\pi\nu_l t},$$

with carrier frequency f_c, subcarrier offsets f, time $t = nT$ of OFDM symbol n (symbol duration $T = 1/\Delta f$ for subcarrier spacing Δf), delay τ_l and Doppler shift ν_l. This is the formula of Sionna's `Paths.cfr`. `cfr_normalize_delays` first subtracts the earliest delay of each channel; `cfr_normalize` scales each channel to unit average energy. The parameters are saved as `cfr_params`.

Sionna computes the carrier phase $2\pi f_c \tau_l$ in float32, so the CSIGen CFR differs from Sionna's by about 1% at 28 GHz and 0.1% at 3.5 GHz. Neither is more accurate, because the delays themselves are float32; the float64 computation makes the CFR reproducible from the saved paths.

6.2 Rebuilding and Modifying the CFR

`load_cfr(file)` returns the file's `h` bit for bit with the NumPy version and the CPU type that saved the file. With another NumPy version or CPU, it can differ in the last bits. It needs only NumPy, not Sionna RT or a GPU, so a dataset of path data can be used on any machine. Its arguments change the CFR without new ray tracing:

Numerology. `num_subcarriers`, `subcarrier_spacing`, `num_ofdm_symbols`, `normalize_delays`, `normalize`, and `symbol_duration` T, for example $1.07/\Delta f$ for a 7% cyclic prefix.

Arrays. `tx_array` and `rx_array` change `num_rows`, `num_cols`, `vertical_spacing` or `horizontal_spacing` (`retarget_arrays`). With a synthetic array, the coefficient of an antenna pair is the centre coefficient times a plane-wave phase, $a_{l,mn} = a_l e^{j2\pi(\mathbf{r}_m^{\mathrm{rx}}\cdot\hat{\mathbf{k}}_l^{\mathrm{r}} + \mathbf{r}_n^{\mathrm{tx}}\cdot\hat{\mathbf{k}}_l^{\mathrm{t}})/\lambda}$, with element positions $\mathbf{r}$ rotated by `tx_orientation` and `rx_orientations` and the directions $\hat{\mathbf{k}}$ of departure and arrival. The centre coefficient is recovered from `paths_a` and the phases of the new arrays applied. The element pattern, polarization and orientation cannot change, since the coefficients include them.

Velocities. `rx_velocities`, one velocity for all users (3 values) or one per row ($[N, 3]$), in m/s (`retarget_velocities`). The scene and the sectors are static, so $\nu_l = \mathbf{v}^{\mathrm{rx}} \cdot \hat{\mathbf{k}}_l^{\mathrm{r}}/\lambda$, and the Doppler shifts are updated with the change of velocity. The users do not move, so this holds within a frame, not over distances in which the paths change.

Paths. (`select_paths`) `exclude_interactions` drops paths with an interaction of the given types (`specular`, `diffuse`, `refraction`, `diffraction`); `max_bounces` drops paths with more interactions (0 keeps the LoS path only); `power_range_db` drops paths more than this many dB below the strongest path of the channel; `max_paths` then keeps the strongest remaining paths. A channel left without paths has a zero CFR; the normalizations apply to the remaining paths.

The array and velocity changes match new Sionna solves with the new settings to about 10^{-6} whenever the solves find the same paths.

```
from csigen.path_data import load_cfr
f = "output/la_1/<run_id>/channel_tx_0.npz"
h = load_cfr(f)                                          # the saved h
h_64 = load_cfr(f, num_subcarriers=64)                   # 64 subcarriers
h_8x8 = load_cfr(f, tx_array=dict(num_rows=8, num_cols=8))
h_v = load_cfr(f, rx_velocities=[20.0, 0.0, 0.0])        # all users at 20 m/s along x
h_los = load_cfr(f, max_bounces=0)                       # LoS path only
```

6.3 Path Statistics

The metadata `path_stats` holds per-row arrays computed from all paths, also when the paths are not saved (only with a synthetic array). Spreads are power-weighted RMS values, azimuth spreads around the circular mean; angles are in the global frame.

- `num_paths`, `path_gain_db` (total power gain), `distance_m` (3D TX–user distance);
- `first_delay_s`, `mean_delay_s`, `delay_spread_s`;
- `k_factor_db`: LoS power over the power of all other paths (NaN without a LoS path, $+\infty$ with the LoS path only);
- `asd_rad`, `zsd_rad`: azimuth and zenith spreads of departure, at the sector (from `paths_phi_t`, `paths_theta_t`); `asa_rad`, `zsa_rad`: the same for arrival, at the user (from `paths_phi_r`, `paths_theta_r`);
- the strongest path: `dominant_power_fraction`, `dominant_tau_s`, `dominant_theta_t`, `dominant_phi_t`, `dominant_theta_r`, `dominant_phi_r`, `dominant_doppler_hz`, `dominant_num_bounces`;
- `max_abs_doppler_hz`, `max_num_bounces`;
- `power_frac_los`, `power_frac_specular`, `power_frac_diffuse`, `power_frac_refraction`, `power_frac_diffraction`: share of the power in the LoS path and in the paths with at least one interaction of each type (a path with two types counts for both).

7 Train/Test Splits

Each row of a file is the channel between one sector and one user, and with `path_solver_per_tx_users_only: false` each user appears in every sector's file. The command `csigen-split` assigns every channel to train, to test or to neither, so that train and test share no users, no sectors, or neither:

```
csigen-split --run output/<scene_name>/<run_id> --out <out_dir> --by bs --test-ratio 0.2 --seed 1
```

7.1 Methods

`users`. A random `--test-ratio` of the user positions is test, with all their channels. Train users can lie next to test users.

`area`. The bounding box of the users is tiled with squares of `--block-size` meters (default 50), and random blocks are test until they hold at least the test ratio of the users (all but one block at most). With `--guard` g, train users within g meters (horizontally) of a test user are dropped.

`sector`. A random test ratio of the sectors, or those given with `--test-ids`, is test. Users appear on both sides. A test sector usually has train sectors on the same base station, which see the same paths with another orientation, so results are more optimistic than with `bs`.

`bs`. As `sector`, for base stations with all their sectors (base station b holds sectors bN_s to $(b+1)N_s - 1$).

`sector+users`, `sector+area`, `bs+users`, `bs+area`. A sector or base-station split and a user split, each with the test ratio r. A channel is train if its sector and user are both train, test if both are test, and dropped otherwise, so train and test share neither. With all users in every file, test keeps about r^2 of the channels and train $(1-r)^2$; use a larger ratio.

`--disjoint-users` makes a `sector` or `bs` split give each user the side of the sector it was sampled for (`rx_serving_tx`) and drop its channels to the other side. For `users`, `sector` and `bs`, the number of test items is the rounded ratio times the number of positions, sectors or base stations, at least one and at most all but one; sectors without channels are not drawn. Users at the same position, which runs before CSIGen 0.3.0 can contain, stay on one side. `--seed` fixes all random choices.

7.2 Output

`<out_dir>/train/` and `<out_dir>/test/` are runs in the usual format. Every sector file is written on both sides, restricted to that side's rows (possibly none), together with its per-row metadata, path statistics and paths; `source_rows` gives each row's index in the source file and `split_side` the side. `users_per_tx` and `total_users` are renamed to `source_users_per_tx` and `source_total_users`, and `metadata.yaml` gets `num_users`. Each side's `split.yaml` records the method and all parameters, the chosen sectors, base stations or blocks (as indices $[i, j]$ from `block_origin`), the users dropped by the guard or for a shared position, the source run, the CSIGen version, the git commit (only for an editable install from a clone, else null), the command and the time of the split, and the channel and user counts per side and per sector.

8 Examples

The figures of this section come from a run of `config/examples/urban/la_28ghz.yaml` (Los Angeles, 28 GHz). Each sector's file of this run holds all users. The run has 20,310 channels in total, and 33% of them are LoS. `docs/figures/make_example_figures.py` makes the figures from the run. Fig. 6 shows the path statistics of all channels. Most channels have a few paths (median 4). Fig. 7 shows the CFR of one NLoS channel with 6 paths and a user at 29 m/s. Its strongest paths arrive from opposite directions with Doppler shifts of about +1.7 and −1.9 kHz, so the channel fades over the 14 symbols.

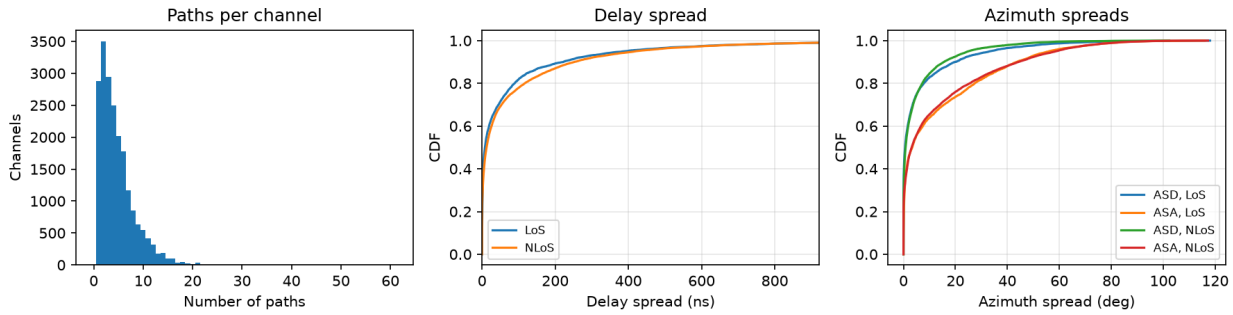


Figure 6: Path statistics of the example run: paths per channel, and CDFs of the delay spread and of the azimuth spreads of departure (ASD) and arrival (ASA), for LoS and NLoS channels.

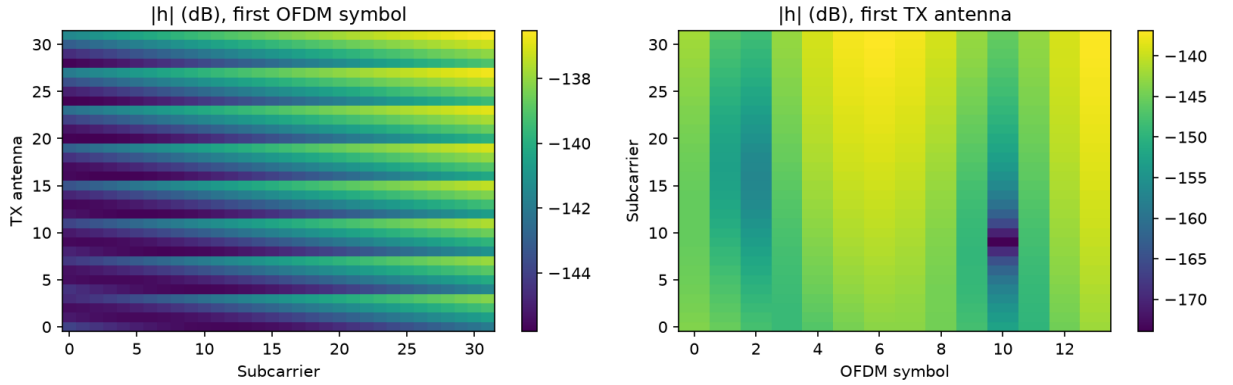


Figure 7: CFR magnitude of one NLoS channel, over TX antennas and subcarriers in the first OFDM symbol (left) and over symbols and subcarriers for the first TX antenna (right).

9 Reproducibility and Limitations

- **Determinism.** Sionna's GPU ray tracing is not bit-identical between runs: two runs of the same config can differ by up to about 1% per channel, occasionally more when a path is found in one run and not the other, and a few radio-map cells near the thresholds can change validity. Each file records the config and the CSIGen version.
- **Precision.** Delays and angles are float32 (Section 6).
- **Memory.** Without a synthetic array, every TX antenna is a separate source, which can exceed the GPU memory after a few sectors. The synthetic array avoids this.
- **Static scene.** Users have velocities but do not move; each channel is one snapshot of `num_ofdm_symbols` symbols.
- **Diffuse scattering.** The included scenes have no diffuse paths (Section 2.2).